Gemini 3.5 Flash Extended

Topic 3.1: Introduction and Transport-Layer Services

Core Concepts

- **Logical Communication**: A transport-layer protocol provides logical communication between application processes running on different hosts. From the application's perspective, it behaves as if the hosts were directly connected, ignoring the underlying physical infrastructure (routers, link types, etc.).
- **End-System Implementation**: Transport protocols are implemented exclusively in the end systems (hosts) and not in network routers.

Packet Journey: Sender vs. Receiver

- **Sending Side**:
  - The transport layer receives application-layer messages.
  - It breaks messages into smaller chunks if necessary and appends a transport-layer header to create a **segment**.
  - The segment is passed to the network layer, where it is encapsulated inside a network datagram.
- **Network Core**: Routers handle only the network-layer fields of the datagram; they do not inspect or modify the encapsulated transport-layer segment.
- **Receiving Side**:
  - The network layer extracts the transport segment from the datagram.
  - The segment is passed up to the transport layer, which processes it and ensures the payload data is available to the destination application process.

Figure 3.1: Logical End-to-End Transport vs. Physical Path

```
text


  +------------------+                            +------------------+
  | Application      |                            | Application      |
  +------------------+                            +------------------+
  | Transport        | <===== Logical End-to-End ===>| Transport        |
  +------------------+        Process Communication  +------------------+
  | Network          |                            | Network          |
  +------------------+                            +------------------+
  | Data Link        |                            | Data Link        |
  +------------------+                            +------------------+
  | Physical         |                            | Physical         |
  +------------------+                            +------------------+
     Host A (Sender)                                 Host B (Receiver)
           \                                               /
            \_______ [Network Core: Routers & Links] _________/
                          Physical Routing Path
```

Relationship Between Transport and Network Layers

While both layers form the core of network communication, their focus areas differ drastically:

- **Network Layer**: Provides logical communication between **hosts**.
- **Transport Layer**: Provides logical communication between **processes** running on those hosts. It relies on and extends the network-layer services.

> **The Household Analogy** Imagine two houses (hosts), one on the East Coast and one on the West Coast, each containing 12 children (processes) who write letters to one another.
>
> - **Cousins** = Application processes
> - **Letters in envelopes** = Application messages
> - **Houses** = Hosts (end systems)
> - **Ann and Bill** (the kids responsible for collecting/distributing mail internally in each house) = Transport-layer protocol
> - **Postal Service** = Network-layer protocol

Service Limitations and Enhancements

- **Layer Constraints**: A transport protocol cannot guarantee characteristics like bandwidth or maximum delay if the underlying network layer cannot provide them.
- **Service Countermeasures**: A transport protocol *can* offer certain characteristics even if the network layer is deficient. For example, it can provide reliable data transfer (via error detection and retransmissions) over an unreliable network layer, or encryption to ensure confidentiality over an insecure network layer.

Overview of Internet Transport Protocols

The Internet network layer uses the **Internet Protocol (IP)**, which offers a **best-effort delivery service**. IP is fundamentally **unreliable** because it does not guarantee packet delivery, orderly delivery, or data integrity. Every host on the network possesses at least one unique IP address.

To bridge this gap, the transport layer provides two primary protocols to applications:

Feature / Service	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
Connection State	**Connectionless**: No handshaking before sending data.	**Connection-oriented**: Requires a three-way handshake.
Reliability	**Unreliable**: No guarantees that data will arrive or arrive in order.	**Reliable**: Guarantees correct, uncorrupted, and in-order delivery.
Primary Services	Process-to-process data delivery (multiplexing/demultiplexing) and basic error checking.	Error checking, flow control, sequence numbers, ACKs, and timers.
Congestion Control	**None**: Unregulated; can blast data at any rate for as long as it wants.	**Built-in**: Prevents a single connection from overwhelming links and routers; regulates sending rate based on network state.
Header Overhead	Small: 8 bytes.	Large: 20 bytes.

Gemini 3.5 Flash Extended

Topic 3.2: Multiplexing and Demultiplexing

Core Concepts

- **Host-to-Host vs. Process-to-Process**: The network layer provides communication between hosts, while the transport layer extends this into a process-to-process delivery service.
- **Sockets**: Intermediary doors through which data passes from the network to an application process, and vice versa. Since a receiving host can have multiple sockets active simultaneously, each socket must have a unique identifier.

Definitions

- **Demultiplexing**: The job of delivering the data within a transport-layer segment to the correct intermediary socket.
- **Multiplexing**: The job of gathering data chunks at the source host from different sockets, encapsulating each chunk with header information (including port numbers) to create segments, and passing them down to the network layer.

Port Numbers and Segment Structure

To implement multiplexing and demultiplexing, segments contain two key fields in their headers:

1. **Source Port Number Field** (16 bits)
2. **Destination Port Number Field** (16 bits)

- **Range**: Port numbers are 16-bit numbers ranging from `0` to `65535`.
- **Well-Known Ports**: Ports from `0` to `1023` are restricted and reserved for well-known application protocols.
  - *Examples*: HTTP uses port `80`, FTP uses port `21`.

```
text

 0                   15 16                  31 bit bounds
+-----------------------+-----------------------+
|   Source Port Number    |  Destination Port Number|
+-----------------------+-----------------------+
|              Other Header Fields                |
+-------------------------------------------------+
|              Application Data                   |
|                  (Message)                      |
+-------------------------------------------------+
```

Connectionless Multiplexing and Demultiplexing (UDP)

- **Socket Identification**: A UDP socket is fully identified by a **two-tuple**:
  $(\text{Destination IP Address}, \text{Destination Port Number})$
- **Demultiplexing Behavior**: When a UDP segment arrives, the host transport layer examines the destination port number and directs it to the corresponding socket.

- **Crucial Characteristic**: If two UDP segments come from different source IP addresses and/or different source port numbers but share the *same* destination IP address and destination port number, they are routed to the **same destination socket** and process.
- **Purpose of Source Port**: It acts as part of a "return address". When the receiver wants to reply, the destination port of its outgoing segment is copied directly from the source port of the incoming segment.

Connection-Oriented Multiplexing and Demultiplexing (TCP)

- **Socket Identification**: A TCP socket is uniquely identified by a **four-tuple**:
  $$(\text{Source IP Address}, \text{Source Port Number}, \text{Destination IP Address}, \text{Destination Port Number})$$
- **Demultiplexing Behavior**: The host uses all four values to route a segment to its designated socket.
- **Crucial Characteristic**: In contrast to UDP, two arriving TCP segments with different source IP addresses or different source ports will be directed to **two entirely different sockets**—even if they share the exact same destination port.
- **The Welcoming Socket**: A TCP server process maintains a "welcoming socket" listening on a specific port (e.g., port 12000 or port 80).
  1. The client initiates a connection request (a TCP segment with a special setup bit).
  2. When the server OS receives this on the welcoming port, it locates the process and accepts the connection.
  3. The server process spawns a **new socket** mapped specifically to that connection's unique 4-tuple. All future segments matching those four parameters traverse this newly dedicated socket.

Web Servers and Sockets

High-performing modern Web servers typically do not create a separate process for each connection because of overhead. Instead, they run a single process and spawn a new lightweight **thread** with its own distinct connection socket for each client.

- **Persistent HTTP**: The client and server exchange multiple request/response messages over the *same* server socket throughout the connection's lifespan.
- **Non-Persistent HTTP**: A brand new TCP connection—and thus a new socket—is created and closed for *every* individual request and response, which can severely stress a busy server.

Focus on Security: Port Scanning

- **Concept**: Since open ports map directly to specific network applications (e.g., port 1434 to MS SQL Server), determining which ports are open allows administrators (or attackers) to map out what software a host is running.
- **Vulnerability Mapping**: If a host runs an application with a known security flaw (such as a buffer overflow), finding that open port exposes the system to exploitation.
- **Tools**: Public domain utilities called **port scanners** sequentially probe ports to identify status. The most widely used tool is `nmap`:
  - *For TCP*: `nmap` sequentially scans ports looking for those accepting TCP connection requests.
  - *For UDP*: It scans looking for responses to transmitted UDP segments.
  - *Results*: It outputs whether each targeted port is *open*, *closed*, or *unreachable*.

Gemini 3.5 Flash Extended

Topic 3.3: Connectionless Transport: UDP

Core Concepts

- **No-Frills Protocol**: UDP (User Datagram Protocol), defined in RFC 768, is a minimal, no-frills transport protocol. Aside from providing a basic multiplexing/demultiplexing function and light error checking, it adds nothing to the core network-layer capabilities of IP.
- **Direct IP Interaction**: When an application developer selects UDP instead of TCP, the application layer is communicating almost directly with the underlying IP protocol. UDP simply takes application messages, appends source/destination port fields, adds two other minor fields, and hands the segment to the network layer.
- **Connectionless State**: There is absolutely no handshaking between the sending and receiving transport-layer entities before a segment is dispatched. Because of this, UDP is classified as a connectionless protocol.

Why Choose UDP Over TCP?

While TCP guarantees reliable data transfer and UDP does not, many application developers deliberately choose UDP for the following strategic reasons:

1. **Finer Application-Level Control Over Data Transmission**:
    - Under UDP, the instant an application passes a data chunk down, UDP encapsulates it and immediately transmits it to the network layer.
    - TCP, by contrast, has a congestion-control mechanism that throttles the sender when network links become congested. TCP will also continuously retransmit a segment until its receipt is acknowledged, regardless of how long that takes. Real-time applications require steady minimum sending rates and can tolerate a small amount of data loss, making TCP's congestion throttling and retransmission delays highly detrimental.
2. **No Connection Establishment Delay**:
    - TCP utilizes a mandatory three-way handshake prior to any data exchange, introducing introductory propagation delays. UDP transmits immediately without any formal preliminaries.
    - This zero-setup delay is the primary reason why **DNS (Domain Name System)** runs over UDP; running it over TCP would significantly slow down Internet lookups. (Note: The modern **QUIC protocol** used in browsers like Google Chrome utilizes UDP as its base to bypass connection delays, implementing custom reliability directly at the application layer).
3. **No Connection State**:
    - TCP end systems must track and maintain active connection states, which include receive/send buffers, congestion-control parameters, and sequence/acknowledgment numbers.
    - UDP does not maintain connection states or track any of these parameters. As a result, a server running an application over UDP can generally support significantly more active clients simultaneously than a server running over TCP.
4. **Minimal Packet Header Overhead**:
    - Every single TCP segment carries a minimum of 20 bytes of header overhead, whereas a UDP segment requires only 8 bytes of header overhead.

Popular Internet Applications and Their Transport Protocols

The table below highlights how transport protocol selection aligns with application requirements:

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic Mail	SMTP	TCP
Remote Terminal Access	Telnet	TCP
Web	HTTP	TCP
File Transfer	FTP	TCP
Remote File Server	NFS	Typically UDP
Streaming Multimedia	Typically Proprietary	UDP or TCP
Internet Telephony	Typically Proprietary	UDP or TCP
Network Management	SNMP	Typically UDP
Name Translation	DNS	Typically UDP

> **The UDP Multimedia Controversy** Running high-bit-rate multimedia streaming over UDP is highly controversial in networking. Because UDP lacks congestion control, massive concurrent streams can cause serious router buffer overflows, resulting in high packet loss rates. Furthermore, this unregulated traffic crowds out cooperative TCP sessions, forcing TCP senders to aggressively throttle down their rates.

UDP Segment Structure

The UDP header is exceptionally stream-lined, containing exactly four fields, each consisting of 2 bytes (for a total header overhead of 8 bytes):

1. **Source Port Number**: Identifies the sending process and forms part of the "return address".
2. **Destination Port Number**: Examined by the receiving host's transport layer to deliver the segment's data payload to the correct intermediary socket.
3. **Length**: Specifies the total size of the UDP segment (header plus data) in bytes. This is explicitly required since the size of the application data payload varies from segment to segment.
4. **Checksum**: Used by the receiving host to perform end-to-end error detection.

```text
 0                   15 16                  31 bit bounds
+-----------------------+-----------------------+
|   Source Port Number  | Destination Port Number |
+-----------------------+-----------------------+
|        Length         |       Checksum        |
+-----------------------+-----------------------+
|                                               |
|                Application Data               |
|                   (Message)                   |
|                                               |
+-----------------------------------------------+
```

UDP Checksum & The End-to-End Principle

The checksum field provides error detection to determine whether bits within the UDP segment have been altered (e.g., by electrical noise on a link or while buffered inside a router).

1. Sender-Side Calculation

The sender splits the segment into 16-bit words and performs a **1s complement sum**. Any overflow bit generated during addition is wrapped around and added to the lowest-order bit. The sender then takes the 1s complement of this final sum (converting all 0s to 1s and 1s to 0s) and writes it into the checksum field.

- **Example**: Suppose we want to sum the following three 16-bit words:

```
text

  0110011001100000
+ 0101010101010101
------------------
  1011101110110101   (Sum of first two words)

+ 1000111100001100   (Add the third word)
------------------
1 0100101011000010   (Overflow detected!)
+               1    (Wrap around the overflow bit)
------------------
  0100101011000011   (Final wrapped sum)
```

  Flipping all the bits (1s complement) of `0100101011000011` yields **`1011010100111100`**, which is saved as the checksum.

2. Receiver-Side Verification

The receiver adds all 16-bit words together, including the checksum field itself. If no errors were introduced, the total sum at the receiver will cleanly equal all 1s (`1111111111111111`). If even a single bit evaluates to `0`, the segment is known to be corrupted. UDP does not attempt error recovery; it will either silently discard the damaged segment or forward it to the application with an explicit warning.

3. Why Have a Transport-Layer Checksum if the Link-Layer Checks for Errors?

Many link-layer protocols (like Ethernet) feature built-in error checking. However, UDP must still provide a checksum due to the **End-to-End Principle** of system design:

- **No Path Guarantees**: There is no guarantee that every intermediate link along the multihop path between source and destination hosts implements an error-checking protocol. One of the links might be entirely unprotected.
- **In-Memory Router Errors**: Even if packets are safely guarded across individual links, bit errors can spontaneously occur due to hardware faults while a segment is sitting inside a router's memory buffer.
- **Core Rule**: Since end-to-end data integrity cannot be fully guaranteed by lower layers, the transport layer must implement error detection as a vital safety measure.

Gemini 3.5 Flash Extended

Topic 3.4: Principles of Reliable Data Transfer

---

1. Conceptual Framework & Service Abstraction

The problem of implementing reliable data transfer occurs not only at the transport layer, but at the link layer and application layer as well, making it one of the most fundamentally important problems in networking.

- **Service Abstraction Provided**: To the upper-layer entity, the protocol provides a **reliable channel**. No data bits are corrupted (flipped from 0 to 1 or vice versa) or lost, and all packets are delivered in the exact order in which they were sent.
- **The Layer Core Challenge**: The layer *beneath* the reliable data transfer protocol is often inherently **unreliable**. For example, TCP builds a reliable data transfer service on top of an unreliable, best-effort network layer (IP).

Protocol Interfaces (APIs)

- `rdt_send()` : Invoked from above by the upper-layer application to pass data to the sending side of the reliable data transfer protocol.
- `udt_send()` : Called by the `rdt` protocol to transmit a packet over an **unreliable channel** to the other side.
- `rdt_rcv()` : Called when a packet arrives at the receiving side of the channel.
- `deliver_data()` : Invoked by the receiving side of the `rdt` protocol to pass correctly received data up to the application layer.

*Note: For explanation purposes, we consider **unidirectional data transfer** (data flowing from sender to receiver), though control packets (ACKs/NAKs) must flow in both directions.*

---

2. Building a Reliable Data Transfer Protocol (Incremental Design)

Protocol rdt1.0: Transfer over a Perfectly Reliable Channel

- **Assumptions**: The underlying channel is completely reliable. Packets are delivered perfectly, in-order, with zero corruption or loss. The receiver can process data as fast as it arrives.
- **Mechanisms**: No feedback or recovery is required. The Finite-State Machines (FSMs) contain only a single state:
  - *Sender*: Waits for a call from above, encapsulates data via `make_pkt(data)` , and sends it via `udt_send(packet)` .
  - *Receiver*: Waits for a call from below, extracts data via `extract(packet, data)` , and passes it up via `deliver_data(data)` .

Protocol rdt2.0: Transfer over a Channel with Bit Errors

- **Assumptions**: Packets can suffer bit corruption during transmission or buffering, but packets are never entirely lost.
- **Core Concept (ARQ Protocols)**: Known as *Automatic Repeat reQuest* protocols, they mimic human telephone dictation using three new baseline capabilities:
  1. **Error Detection**: A checksum field is added to allow the receiver to detect bit changes.
  2. **Receiver Feedback**: Positive Acknowledgments (**ACK** / "OK") or Negative Acknowledgments (**NAK** / "Please repeat that") are sent back.
  3. **Retransmission**: If a packet is received in error, the sender re-sends the last data packet.
- **Behavior Type**: Classified as a **stop-and-wait protocol**. The sender transitions to a `Wait for ACK or NAK` state and refuses to accept new data from the application until it receives a valid ACK.

> ⚠️ **The Fatal Flaw of rdt2.0** `rdt2.0` does not account for the possibility that the **ACK or NAK packet itself becomes corrupted**. If the sender receives a garbled feedback packet, it cannot know if the receiver safely recorded the data or is asking for a repeat. Simply retransmitting introduces a dangerous side effect: **duplicate packets**. The receiver cannot distinguish whether a newly arrived packet is fresh data or an unannounced duplicate.

Protocol rdt2.1: Handling Corrupted ACKs/NAKs via Sequence Numbers

- **The Fix**: A **1-bit sequence number** (alternating between `0` and `1`) is written into the header of every data packet.

- **Sender State Doubling**: The sender FSM doubles its states to track whether it is currently transmitting data packet 0 or data packet 1.
- **Receiver Actions**:
  - If a received packet is corrupted, the receiver sends a NAK.
  - If a packet arrives clean but has a duplicate sequence number (e.g., expecting `1` but getting `0`), the receiver knows its last ACK was corrupted or lost at the sender. It discards the duplicate packet but **re-sends a positive ACK** to force the sender to move forward.

Protocol rdt2.2: A NAK-Free Alternative

- **The Fix**: We can achieve the exact same control effect as a NAK by using **ACKs labeled with sequence numbers**.
- Instead of sending a NAK when a packet is corrupted or out of order, the receiver simply transmits an ACK for the *last correctly received, in-order packet*.
- When a sender transmits packet `1`, but receives a duplicate ACK labeled explicitly for packet `0`, it serves as an implicit NAK indicating that packet `1` was not received correctly.

Protocol rdt3.0: Channels with Both Bit Errors and Packet Loss

- **Assumptions**: Packets can now be corrupted *and* completely lost inside the channel network.
- **The Fix (Countdown Timer)**: The burden of detection moves to the sender. The sender sets a countdown timer whenever a packet is dispatched.
- **Timeout Event**: If a data packet or its corresponding ACK is completely dropped, the sender will experience a timeout and **retransmit** the segment.
- **Premature Timeouts**: If a packet is merely heavily delayed but not lost, a timeout occurs early, resulting in a duplicate packet. The sequence numbers native to `rdt2.2` seamlessly handle this redundancy.
- **Alias**: Known as the **Alternating-Bit Protocol** due to its sequence numbers flipping cleanly between 0 and 1.

```text
[Sender]                                    [Receiver]
   | -------- pkt0 (Seq=0) -------->             |
   |                                             | (Processes pkt0)
   | <------- ACK0 -----------------             | (Sends ACK0)
   |                                             |
   | -------- pkt1 (Seq=1) -------->             |
   |            X (PACKET LOST)                  |
   |                                             |
   | [Timer Expires: Timeout!]                   |
   | -------- pkt1 (Seq=1) -------->             |
   |                                             | (Processes pkt1)
   | <------- ACK1 -----------------             | (Sends ACK1)
```

---

3. Pipelined Reliable Data Transfer

While `rdt3.0` is functionally bulletproof, its stop-and-wait nature creates an extreme performance bottleneck in high-speed, long-distance networks.

The Stop-and-Wait Performance Problem

Consider a link over a cross-country distance with a Round-Trip Time (RTT) of 30 ms, operating at a transmission rate (R) of 1 Gbps. With a packet size (L) of 1,000 bytes (8,000 bits):

$$d_{trans} = \frac{L}{R} = \frac{8{,}000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microseconds}$$

The sender utilization (U_{sender}) evaluates to a dismal fraction:

$$U_{sender} = \frac{d_{trans}}{RTT + d_{trans}} = \frac{0.008 \text{ ms}}{30 \text{ ms} + 0.008 \text{ ms}} = 0.00027$$

The sender is actively transmitting data onto the link for only **0.027%** of the total time, yielding an effective throughput of just 267 kbps on a 1 Gbps pipeline!

The Pipelining Solution

Instead of waiting for an ACK after every packet, the sender is allowed to transmit multiple packets concurrently without immediate acknowledgments, effectively filling the network pipeline.

```
text

Stop-and-Wait:  [Pkt0] ---------- (Wait 1 RTT) -----------> [Pkt1]
Pipelined:      [Pkt0][Pkt1][Pkt2][Pkt3] -> (Continuous streaming of chunks)
```

Pipelining introduces distinct requirements:

- The range of packet sequence numbers must be expanded far beyond a single bit.
- Both the sender and receiver may be required to maintain buffers to store in-flight or out-of-order data packets.

---

4. Sliding-Window Protocols: Go-Back-N (GBN) vs. Selective Repeat (SR)

To recover from lost or corrupted fragments within a pipeline, modern protocols follow one of two standardized architectural paths:

Strategy A: Go-Back-N (GBN)

The sender is allowed to transmit up to N unacknowledged packets simultaneously in a logical window. As older packets are safely acknowledged, the window slides forward over the sequence number space.

- **Cumulative Acknowledgments**: An ACK labeled with sequence number n implies a *cumulative acknowledgment*, stating that all packets up to and including n have been successfully received.
- **Single Retransmission Timer**: The sender maintains only a single hardware timer, mapped directly to the **oldest transmitted but unacknowledged packet** (`base`).
- **Receiver Discard Policy**: The GBN receiver **discards all out-of-order packets**. It refuses to buffer any early arrivals.
  - *Example*: If packet `2` is lost, but packets `3` , `4` , and `5` arrive cleanly, the receiver throws away `3` , `4` , and `5` , and continuously re-sends `ACK 1` .
- **Timeout Execution**: Upon timer expiration, the sender must **Go Back** and retransmit the *entire window of unacknowledged packets* (`base` through `nextseqnum-1`).

Strategy B: Selective Repeat (SR)

Go-Back-N suffers massive performance drops when channel error rates increase because a single dropped packet causes a cascading retransmission of many perfectly healthy packets. Selective Repeat eliminates this waste by forcing the sender to retransmit *only* specific packets that were lost or damaged.

- **Individual Acknowledgments**: The receiver individually acknowledges every cleanly arriving packet, completely independent of order.
- **Receiver-Side Buffering**: Out-of-order packets are held safely in a receiver buffer. When the missing packet ("the gap") finally arrives, a sequential batch of packets is delivered to the application layer, and the receive window slides forward.

- **Logical Timers per Packet**: The sender must maintain a separate logical timer for *each individual packet* to drive selective retransmissions.

> ⚠️ **The Sequence Number vs. Window Size Constraint** In Selective Repeat, the sender and receiver windows do not always move forward in perfect synchronization. If the window size N is too large relative to the total range of available sequence numbers, a catastrophic overlap occurs. The receiver cannot distinguish between a *retransmitted copy of an old packet* (if all ACKs were dropped) versus a *brand-new packet* utilizing a recycled sequence number.
>
> To resolve this structural dilemma, **the window size must be less than or equal to exactly half of the total sequence number space size** for SR protocols:
>
> $N_{\text{window}} \leq \frac{1}{2} \cdot 2^k$

Summary of Reliable Data Transfer Mechanisms

Mechanism	Main Objective / Use Case	Comments
Checksum	Bit Error Detection	Computes bit-level integrity across headers and data payload.
Timer	Packet Loss Recovery	Triggers automatic retransmission when a packet or its ACK is dropped.
Sequence Number	Duplicate Detection & Reordering	Labels packets sequentially so the receiver can detect duplicate segments or identify missing items.
ACK / Feedback	Signaling Successful Arrival	Tells the sender a chunk arrived; can be individual (SR) or cumulative (GBN/TCP).
Pipelining / Window	Performance Optimization	Allows multiple packets to sit in-flight simultaneously, dramatically increasing link utilization.

---

*Ready for the next section? Say **continue** to review the details of the TCP protocol.*

Gemini 3.5 Flash Extended

Topic 3.5: Connection-Oriented Transport: TCP

---

1. The TCP Connection

TCP is the Internet's foundational connection-oriented, reliable transport protocol. It is characterized by several core properties:

- **Connection-Oriented State**: Before applications can exchange data, the two end systems must perform a preliminary "handshake" to initialize active state variables and establish communication parameters.
- **Logical End-to-End Control**: The "connection" is completely logical. State variables and buffers reside exclusively within the software stack of the two communicating hosts. Intermediate network routers and link switches are entirely oblivious to TCP connections; they handle only individual IP datagrams.
- **Full-Duplex Service**: Data can flow concurrently in both directions over a single connection (e.g., from Process A to Process B and from Process B to Process A simultaneously).

- **Point-to-Point Architecture**: A TCP connection is strictly limited to a single sender and a single receiver. Multicasting or broadcasting data from one sender to multiple receivers in a single transaction is impossible.
- **Data Stream Abstraction**: TCP views data as an unstructured, but strictly ordered, continuous stream of bytes.

Buffers and Segments

When an application process writes data through a socket, the data enters the TCP connection's allocated **send buffer**. From there, TCP extracts blocks of data at its own convenience and packages them with a TCP header to form a **segment**.

- **Maximum Segment Size (MSS)**: The maximum limit on the application-layer payload data encapsulated within a single segment.
- **Relationship to MTU**: The MSS is constrained by the link layer's **Maximum Transmission Unit (MTU)**—the largest physical frame a host can transmit onto a link. For standard Ethernet and PPP networks, the MTU is `1,500 bytes`. Since the mandatory combined TCP/IP header length is typically `40 bytes`, a standard Internet connection utilizes an MSS of **`1,460 bytes`** $(1,500 - 40)$.

---

2. TCP Segment Structure

A standard TCP segment is divided into header fields and a data payload field. The typical header layout spanning 32 bits horizontally is structured as follows:

```
text

 0                   15 16                   31 bit bounds
+-----------------------+-----------------------+
|   Source Port Number    | Destination Port Number |
+-----------------------+-----------------------+
|                Sequence Number                  |
+-------------------------------------------------+
|              Acknowledgment Number              |
+-----------------------+-----------------------+
| H.Len | Unused  | Flags |     Receive Window      |
+-----------------------+-----------------------+
|    Internet Checksum    |  Urgent Data Pointer    |
+-----------------------+-----------------------+
|                   Options                       |
+-------------------------------------------------+
|                    Data                         |
|           (Application Payload)                 |
+-------------------------------------------------+
```

Key Header Fields Explained

- **Source & Destination Ports** (16 bits each): Drive transport-layer multiplexing and demultiplexing to match segments with applications.
- **Sequence Number** (32 bits): Represents the **byte-stream number of the first byte** contained within the segment's data payload field. It does *not* count segment packages.
- **Acknowledgment Number** (32 bits): Specifies the sequence number of the **next sequential byte** that the host expects to receive from the remote side.
- **Header Length** (4 bits): Identifies the total size of the TCP header in 32-bit words. This is required because the options field makes the header variable in length (a typical header has an empty options field, defaulting to `20 bytes`).
- **Receive Window** (16 bits): Used to announce the number of bytes the receiver is willing to accept, serving as the basis for TCP flow control.

- **Internet Checksum** (16 bits): Provides end-to-end bit-error detection across the header and data payload.
- **Flags Field** (6 bits active + extension bits):
  - `SYN`: Used during connection establishment to synchronize initial sequence numbers.
  - `ACK`: Indicates that the value carried in the acknowledgment field is valid.
  - `FIN`: Used to cleanly close and tear down a connection.
  - `RST`: Sent to force a connection reset when a segment arrives for an unrecognized socket.
  - `PSH`: Requests that the receiver push data immediately to the application layer without buffering it (rarely used in practice).
  - `URG` & `Urgent Data Pointer`: Signal that a portion of the payload data is marked as urgent (rarely used in practice).
  - `CWR` & `ECE`: Facilitate explicit congestion notification interactions with routers.

Byte-Stream Counting: A Conceptual Example

Suppose a client must transmit a file of `500,000 bytes` with an MSS of `1,000 bytes`, and the initial sequence number happens to be `0`. TCP slices the stream into 500 individual segments:

```
text

Stream Extraction Space:
Byte Indices:  [0 ..................  999] [1000 ................  1999] [2000 ...
               v                          v                           v
Segment Headers:  Seq=0                     Seq=1000                    Seq=2000
```

- **Cumulative Acknowledgments**: If a host receives bytes `0` through `535` cleanly, followed by a gap, and then receives a separate segment containing bytes `900` through `1000`, it will set its outward acknowledgment number to **`536`**. Because TCP only acknowledges data up to the first missing byte in the ordered stream, it is strictly cumulative.
- **Out-of-Order Policy**: When a gap occurs, the TCP specification leaves the handling of out-of-order segments up to the developer. Implementations can either discard them or buffer them. In practice, modern operating systems buffer them to conserve network bandwidth.

---

3. Round-Trip Time (RTT) Estimation and Timeout

TCP uses a time-based retransmission mechanism to recover from lost segments. To prevent unnecessary retransmissions, the timeout interval must be carefully calculated relative to the network's round-trip time.

Step 1: Track SampleRTT

The host measures the `SampleRTT`—the time elapsed between passing a segment to the network layer and receiving an ACK for it. To ensure an accurate model, TCP avoids measuring `SampleRTT` for any segment that has been retransmitted.

Step 2: Compute EstimatedRTT (Exponential Moving Average)

Because network conditions fluctuate, TCP smooths individual samples using an exponential moving average:

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

The IETF recommendation sets the weight factor at $\alpha = 0.125$.

Step 3: Compute RTT Deviation (DevRTT)

TCP estimates how much the `SampleRTT` typically deviates from the estimated average:

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

The recommended weight factor is $\beta = 0.25$.

Step 4: Calculate TimeoutInterval

The final safety margin is computed using both values:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

When a timeout occurs, TCP politely doubles the `TimeoutInterval` (exponential backoff) to avoid exacerbating network congestion. As soon as a fresh segment is successfully acknowledged, the standard formula is reapplied.

---

4. Reliable Data Transfer Mechanics

TCP builds a reliable data transport service on top of IP's unreliable best-effort network service. To conserve processing overhead, TCP relies on a **single retransmission hardware timer** mapped directly to the oldest unacknowledged segment (`SendBase`).

Simplified Operational FSM Loop

```
text

+-----------------------------------------------------------+
|                      LOOP FOREVER                         |
+-----------------------------------------------------------+
       |
       +---> [Event: Data Received From App Above]
       |     - Create segment with Sequence Number = NextSeqNum
       |     - Pass segment to IP network layer
       |     - IF (Timer is not currently running) -> Start Timer
       |     - NextSeqNum = NextSeqNum + Length(Data)
       |
       +---> [Event: Retransmission Timer Expires]
       |     - Retransmit the segment with SMALLEST unacknowledged sequence number
       |     - Restart Timer with exponential backoff interval
       |
       +---> [Event: Valid ACK Segment Arrives (ACK value = y)]
             - IF (y > SendBase)
                  * SendBase = y  (Slide window forward)
                  * IF (there are remaining unacknowledged segments) -> Restart Timer
                  * ELSE -> Stop Timer
```

Fast Retransmit Optimization

Waiting for a standard timer to expire can introduce substantial delays in high-speed networks. Because a sender often transmits multiple segments back-to-back, a lost segment typically creates a gap that prompts the receiver to send multiple duplicate ACKs.

- **The Rule**: If the TCP sender receives **three duplicate ACKs** for the same data sequence number (which means 4 identical ACKs in total), it takes this as an implicit message that the next segment was lost.
- **Action**: It immediately executes a **Fast Retransmit**, resending the missing segment before its structural countdown timer expires.

```
text

Sender                                    Receiver
  | -------- Seq=100 (Data) ------->          |  (Arrives safely)
  | -------- Seq=120 (Lost!) ------> X        |  (Dropped packet)
```

```
| -------- Seq=140 (Data) ------->            |  (Gap detected! Sends ACK 120)
|                                <-- ACK 120 -|
| -------- Seq=160 (Data) ------->            |  (Gap remains! Sends Duplicate ACK 120)
|                                <-- ACK 120 -|
| -------- Seq=180 (Data) ------->            |  (Gap remains! Sends Duplicate ACK 120)
|                                <-- ACK 120 -| -> [3rd Duplicate ACK Received!]
|                                             |
| ====== FAST RETRANSMIT: Resends Seq=120 ===> |
```

Architectural Hybrid Status

Is TCP classified as a Go-Back-N (GBN) or Selective Repeat (SR) protocol?

- It resembles **GBN** because it tracks window bounds using structural variables (`SendBase` , `NextSeqNum`) and utilizes a single cumulative acknowledgment scheme.
- It resembles **SR** because modern implementations buffer out-of-order data segments and retransmit at most a single missing segment upon error, avoiding cascading window re-transmissions.
- **Conclusion**: TCP is best categorized as a custom **hybrid protocol**.

---

5. Flow Control Service

Flow control is a speed-matching service that balances the sender's transmission rate against the rate at which the receiving application process reads data. This eliminates the possibility of the sender overflowing the receiver's buffer space.

The Mathematical Window Calculations

The receiver allocates a buffer space of size `RcvBuffer` to the connection. The receiver's transport stack tracks two primary variables:

- `LastByteRcvd` : The sequence boundary of the last byte to arrive from the network and enter the buffer.
- `LastByteRead` : The sequence boundary of the last byte read out of the buffer by the local application process.

To prevent buffer overflow, the operating system ensures that:

$$\text{LastByteRcvd} - \text{LastByteRead} \leq \text{RcvBuffer}$$

The dynamic variable **`rwnd`** **(Receive Window)** represents the remaining spare room available:

$$\text{rwnd} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

Sender Enforcement Constraint

The receiver constantly advertises its current `rwnd` value inside the receive window header field of every segment it transmits back to the sender. The sender tracks its own variables (`LastByteSent` , `LastByteAcked`) and ensures that:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

The Zero-Window Deadlock Problem & Solution

If the receiver's buffer fills completely, it advertises `rwnd = 0` , which blocks the sender from transmitting further data. If the receiving application later clears the buffer but has no data or acknowledgments to send back, the client remains blocked indefinitely.

- **The Solution**: When `rwnd = 0` , the sender is required to continue transmitting **1-byte probe segments** to the receiver. These probes force the receiver to reply with an updated acknowledgment header containing the latest non-zero `rwnd` value.

---

6. TCP Connection Management

The Three-Way Handshake (Connection Setup)

Before data transmission can begin, the client and server exchange three preliminary segments to synchronize sequence numbers and verify readiness.

```
text

Client Host                                          Server Host
    |                                                     |
    | ------------ STEP 1: SYN Segment -------------->     |
    |      Flags: SYN=1, ACK=0                            |
    |      Seq = client_isn (randomly chosen)             |
    |      Payload: Empty                                 |
    |                                                     |
    | <----------- STEP 2: SYNACK Segment ------------    |
    |      Flags: SYN=1, ACK=1                            |
    |      Seq = server_isn (randomly chosen)             |
    |      Ack = client_isn + 1                           |
    |      Payload: Empty                                 |
    |                                                     |
    | ------------ STEP 3: ACK Segment -------------->    |
    |      Flags: SYN=0, ACK=1                            |
    |      Seq = client_isn + 1                           |
    |      Ack = server_isn + 1                           |
    |      Payload: May encapsulate data payload          |
```

Why choose random Initial Sequence Numbers (ISNs)? Randomization minimizes the risk that a delayed segment from an older, terminated connection is mistaken for a valid, in-flight packet in a newer connection utilizing the same port tuple.

Connection Teardown (Four-Step Termination)

Either entity can choose to terminate communication. When an application process issues a close command, its transport stack triggers a symmetric four-step teardown:

```
text

Host A (Initiator)                                   Host B (Receiver)
    |                                                     |
    | ------------ STEP 1: FIN Segment -------------->    | [Enters CLOSE_WAIT state]
    |      Flags: FIN=1, ACK=1                            |
    |                                                     |
    | <----------- STEP 2: ACK Segment ------------ ----  |
    |      Flags: ACK=1                                   | [Flushes remaining data]
    |                                                     |
    | <----------- STEP 3: FIN Segment -----------------  | [Enters LAST_ACK state]
    |      Flags: FIN=1, ACK=1                            |
    |                                                     |
    | ------------ STEP 4: ACK Segment -------------->    |
    |      Flags: ACK=1                                   | [Deallocates resources]
    |                                                     |
    v                                                     v
 [Enters TIME_WAIT state]                              (CLOSED)
 (Waits 30-120 seconds, then closes)
```

- **Purpose of TIME_WAIT State**: The initiator remains in `TIME_WAIT` for a brief period (typically 30 to 120 seconds) before fully closing. This safety margin ensures that if the final `ACK` is lost, Host A can retransmit it in response to a resent

`FIN`, allowing the connection to close cleanly without leaving half-open state hazards.

Focus on Security: Vulnerabilities & Countermeasures

1. The SYN Flood Attack

- **Concept**: During Step 2 of the handshake, the server must allocate memory buffers and state variables immediately upon receiving an initial `SYN` segment.
- **Exploitation**: Attackers can exploit this behavior by sending a flood of bogus `SYN` segments with spoofed source IP addresses. The server allocates resources for each request and sends a `SYNACK`, but the final `ACK` never arrives. This leaves thousands of "half-open" connections active, exhausting the server's memory and denying service to legitimate users.
- **Countermeasure (SYN Cookies)**: The server avoids allocating any memory or buffer space when a `SYN` arrives. Instead, it uses a secret hash function to generate a unique cryptographic value based on the connection's IP addresses and port numbers, and places this value directly into the `server_isn` field of its `SYNACK` reply. When a legitimate client responds with its final `ACK`, the acknowledgment number field will equal this value plus one. The server runs the hash function again; if the result matches the incoming acknowledgment value, it verifies the client's legitimacy and securely allocates resources all at once.

2. Reset Segment Exploration (Port Scanning)

- If an attacker or port scanner sends a `SYN` segment to a closed port on a target host, the target host's operating system responds with a special reset segment with the flag `RST=1`. This explicit message tells the source that no socket is open on that port, allowing port scanners to quickly map active network services.

---

*Ready for the next section? Say **continue** to review the principles of network congestion control and the TCP congestion algorithm.*

Gemini 3.5 Flash Extended

Topic 3.6: Principles of Congestion Control

Core Concepts

- **Congestion Core Cause**: While packet loss typically presents as a reliable data transfer problem, it is fundamentally a symptom of **router buffer overflow** caused by network congestion.
- **The Problem**: Congestion happens when too many sources attempt to transmit data at rates higher than the network infrastructure can handle.
- **Throttling Mechanism**: While retransmission treats the immediate transport-layer symptom (replacing a lost segment), congestion control mechanisms are required to directly throttle senders to treat the root cause.

---

1. The Causes and the Costs of Congestion (Three Scenarios)

To fully map the behavior and costs of a gridlocked network, three increasingly complex architectural scenarios are examined:

Scenario 1: Two Senders, a Router with Infinite Buffers

- **Setup**: Two hosts (A and B) transmit unique original data at an average rate of λ_{in} bytes/sec over a shared single-hop path containing a single router. The outgoing shared link has a physical capacity of R. The router is assumed to possess an infinite buffer space.
- **Throughput Behavior**: When the host sending rate ranges between 0 and $R/2$, the output throughput (λ_{out}) perfectly tracks the input rate. However, because the link capacity is split equally, the throughput caps asymptotically at exactly $R/2$, regardless of how much higher the senders blast their transmission rates.
- **Delay Behavior**: As the host sending rate approaches $R/2$, the queue inside the infinite buffer swells. Once the sending rate crosses $R/2$, the packet queue grows without bound, and the average queuing delay becomes **infinite**.

> 🔴 **Cost of Congestion #1**: Senders experience massive, compounding queuing delays as the packet-arrival rate nears the ultimate link capacity.

Scenario 2: Two Senders and a Router with Finite Buffers

- **Setup**: The same two-host topology is modified with a real-world constraint: the router buffer space is **finite**. Packets arriving when the buffer is entirely full are instantly **dropped (lost)**. The transport layer is reliable, meaning dropped packets will eventually be retransmitted.
- ** offered load (λ'_{in})**: The total rate at which the transport layer injects segments into the network (original data λ_{in} *plus* retransmitted data).
- **The Performance Costs**:
  1. *Ideal Case (Magical Optimization)*: If the sender could magically sense buffer space and send *only* when a slot was free, no packets would be dropped ($\lambda_{in} = \lambda'_{in}$).
  2. *Retransmit-On-Loss Case*: If the sender retransmits strictly when a packet is known for certain to be lost, a portion of the shared link capacity is permanently consumed by retransmitted data. For example, when the offered load is $R/2$, the throughput of original data delivered to the application layer drops to $R/3$, while the remaining $R/6$ of link space is wasted on retransmitted data.
  3. *Premature Timeout Case*: If a sender times out early due to large queuing delays, it retransmits a segment that is still sitting safely in a router queue. The receiver gets two identical copies, handles the original, and throws away the duplicate retransmission. The capacity used by the router to forward that duplicate copy is completely wasted.

> 🔴 **Cost of Congestion #2**: Senders are forced to dedicate valuable transmission capacity to retransmissions to compensate for packets dropped due to buffer overflows.
>
> 🔴 **Cost of Congestion #3**: Premature timeouts cause unneeded retransmissions, forcing routers to waste link bandwidth forwarding duplicate packets that the receiver will ultimately discard.

Scenario 3: Four Senders, Routers with Finite Buffers, and Multihop Paths

- **Setup**: Four hosts transmit packets over overlapping, two-hop paths through a network of finite-buffered routers. Link capacities are fixed at R bytes/sec.
- **Throughput Collapse**: For tiny values of input traffic (λ_{in}), overflows are rare and throughput tracks input rate cleanly. However, if λ_{in} is expanded to extreme levels across all hosts, a devastating bottleneck occurs.
- *Example*: Host A's traffic to Host C passes through router R1 to router R2. At router R2, A's traffic must aggressively compete for finite buffer slots with traffic from Host B going to Host D. As B's offered load approaches infinity, every empty buffer slot at R2 is instantaneously snatched up by a packet from B. The amount of A's traffic that successfully passes through R2 drops directly to **zero**, causing A's end-to-end throughput to completely collapse.

> 🔴 **Cost of Congestion #4**: When a packet is dropped at a downstream multi-hop router, all the transmission capacity used by upstream routers to forward that packet to that point is completely wasted.

2. General Approaches to Congestion Control

Modern network architectures regulate congestion using one of two primary architectural strategies, classified by whether or not the network layer provides explicit assistance to the transport layer:

Strategy A: End-to-End Congestion Control

- **No Network Assistance**: The network layer provides absolutely zero explicit feedback or support to the transport layer for congestion purposes.
- **Inference-Based**: Senders must independently infer the presence of network congestion by observing end-to-end protocol behavior, such as packet timeouts or receiving a sequence of three duplicate acknowledgments.
- **Internet Default**: The default implementations of IP and TCP use this approach. IP is not required to provide feedback regarding internal queue states; when a segment is dropped due to an over-congested router buffer, TCP detects the loss event and decreases its window size accordingly. (Advanced variants can also monitor progressive round-trip segment delays to infer imminent congestion).

Strategy B: Network-Assisted Congestion Control

- **Explicit Router Feedback**: Network routers provide direct, explicit feedback to the transport endpoints regarding the congestion state of intermediate links.
- **Feedback Methods**:
  1. *Direct Notification (Choke Packets)*: A congested router generates a custom control packet and transmits it directly back to the sender, signaling it to reduce its rate.
  2. *Router Marking (In-Band Signaling)*: A congested router marks or updates a specialized field within a packet that is already flowing from the sender to the receiver. When the receiver opens the marked packet, it notes the congestion tag and transmits a notification back to the sender. This method takes exactly one full round-trip time to alert the source.

---

*Ready for the final section of the chapter? Say **continue** to review the specific mechanics of the TCP Congestion Control Algorithm (AIMD, Slow Start, Congestion Avoidance, and Fast Recovery).*

Gemini 3.5 Flash Extended

Topic 3.7: TCP Congestion Control

---

1. Conceptual Framework & Core Variables

Because the underlying IP network layer provides no explicit, mandatory feedback regarding internal queue bottlenecks, TCP must rely on an **end-to-end congestion control strategy**. Senders independently alter their transmission pace based on observed network loss and acknowledgment timing.

The Congestion Window (`cwnd`)

To regulate traffic flow, a TCP sender maintains an internal state variable known as the **congestion window (`cwnd`)**. It imposes a direct boundary on the maximum volume of unacknowledged data allowed in transit:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{rwnd})$$

Assuming the receiver's buffer space (rwnd) is sufficiently large to be ignored, the sender's dynamic pipeline capability is strictly throttled by cwnd.

Effective Sending Rate

At the start of every Round-Trip Time (RTT), the sender releases cwnd bytes of data into the link pipeline. Upon receiving corresponding acknowledgments one RTT later, the window slides forward. Therefore, the effective transport sending rate is roughly defined as:

$$\text{Transmission Rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

Self-Clocking

When acknowledgments arrive at a high frequency, the window scales up quickly; conversely, if the path has high delay or limited bandwidth, ACKs arrive slowly, and the window grows more gradually. Because TCP uses the arrival rate of ACKs to pace its window expansion, it is classified as a **self-clocking** protocol.

---

2. Perceiving Congestion: Loss Events

A TCP sender identifies network distress through a **loss event**, which is triggered by one of two scenarios:

1. **Retransmission Timeout (RTO) Expiration**: A segment or its ACK is entirely dropped, prompting the hardware countdown timer to expire. This indicates a severe bottleneck or empty buffer drops along the path.
2. **Receipt of Three Duplicate ACKs**: The receiver detects a missing segment in the byte stream but continues to receive subsequent out-of-order packets. It generates a duplicate ACK for each subsequent packet to signal the lower bound of the missing gap. Receiving three duplicate ACKs (4 identical ACKs in total) indicates that the network is still delivering some data, suggesting a less severe form of congestion.

---

3. The TCP Congestion-Control Algorithm (RFC 5681)

The complete congestion management algorithm consists of three primary phases: **Slow Start**, **Congestion Avoidance**, and **Fast Recovery**.

Phase 1: Slow Start

When a connection initializes, `cwnd` begins at a small value of **1 MSS** (Maximum Segment Size).

- **Growth Policy**: For every fresh packet acknowledged, `cwnd` is increased by exactly 1 MSS.
- **The Result**: Because a single RTT accommodates a full window's worth of ACKs, the window size **doubles every RTT**, resulting in rapid **exponential growth**.

```
text

Round 1: Send 1 Segment  ---> Receive 1 ACK  ---> cwnd becomes 2 MSS
Round 2: Send 2 Segments ---> Receive 2 ACKs ---> cwnd becomes 4 MSS
Round 3: Send 4 Segments ---> Receive 4 ACKs ---> cwnd becomes 8 MSS
```

- **Exit Rules**:
    1. *Timeout*: `cwnd` drops back to 1 MSS, the slow-start threshold variable (`ssthresh`) is set to $\frac{\text{cwnd}}{2}$, and Slow Start restarts from scratch.
    2. *Reaching Threshold*: When `cwnd` matches or exceeds `ssthresh` , the protocol transitions to **Congestion Avoidance**.
    3. *Triple Duplicate ACKs*: Executes a fast retransmit, halves the window, and moves into **Fast Recovery**.

Phase 2: Congestion Avoidance

Once `cwnd` surpasses `ssthresh`, exponential growth stops to prevent overwhelming the network. Instead, TCP adopts a cautious **linear increase policy**, adding just **1 MSS per RTT**.

- **Implementation**: Upon every arriving valid ACK, the window scales up fractionally: $\text{cwnd} = \text{cwnd} + \text{MSS} \cdot \left(\frac{\text{MSS}}{\text{cwnd}}\right)$ If the current window holds 10 segments, each individual ACK adds $\frac{1}{10}$ of an MSS. After all 10 acknowledgments return over an RTT, the window increases by exactly 1 MSS.
- **Exit Rules**:
  - *Timeout Event*: `ssthresh` is set to $\frac{\text{cwnd}}{2}$, `cwnd` drops to 1 MSS, and the protocol transitions back to Slow Start.
  - *Triple Duplicate ACK Event*: `ssthresh` is set to $\frac{\text{cwnd}}{2}$, and `cwnd` is set to $\text{ssthresh} + 3\ \text{MSS}$ (inflated to account for the segments that have already cleared the bottleneck). The protocol then enters **Fast Recovery**.

Phase 3: Fast Recovery (Recommended)

During this state, `cwnd` increases by 1 MSS for each additional duplicate ACK received, allowing the sender to transmit new segments if the window constraints permit.

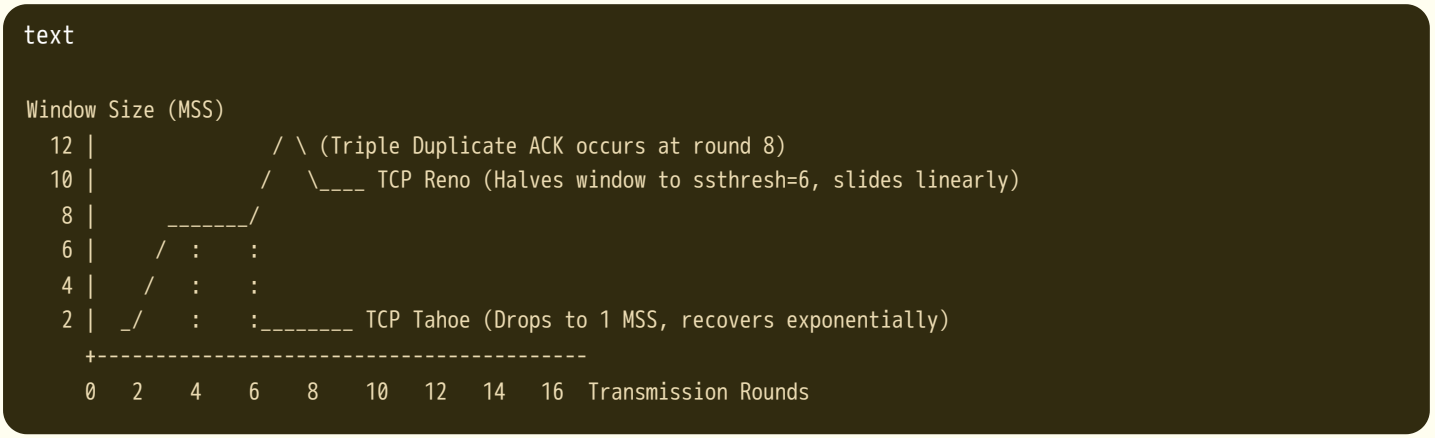
- When a fresh ACK arrives that acknowledges the missing segment and fills the gap, TCP deflates the window back to `ssthresh` and returns to **Congestion Avoidance**.
- If an RTO timeout occurs while in Fast Recovery, it sets `cwnd = 1 MSS`, halves `ssthresh`, and drops back into Slow Start.

---

4. Evolution: TCP Tahoe vs. TCP Reno

The historical evolution of TCP's congestion response highlights key architectural optimizations:

- **TCP Tahoe (Early Version)**: Treated all loss events identically. Whether a timeout or triple duplicate ACKs occurred, Tahoe dropped `cwnd` to 1 MSS and restarted via Slow Start.
- **TCP Reno (Modern Version)**: Introduced the **Fast Recovery** optimization. It distinguishes between the two loss events, avoiding an abrupt drop to 1 MSS during a triple duplicate ACK event because the network is still delivering data.



---

5. Macroscopic Throughput Analysis (AIMD)

Ignoring the brief initial slow-start period, TCP operates primarily via an **Additive-Increase, Multiplicative-Decrease (AIMD)** pattern. This produces a distinct **sawtooth** utilization graph as the sender continuously probes for available bandwidth.

The Steady-State Mathematical Throughput

Let W represent the window size in bytes at the instant a triple duplicate ACK loss event occurs. The window immediately cuts in half to $W/2$ and then climbs linearly by 1 MSS per RTT back to W. Assuming RTT remains stable, the average steady-

state throughput is calculated as:

$$\text{Average Throughput} = 0.75 \cdot \frac{W}{\text{RTT}}$$

The High-Bandwidth Path Problem

On high-speed, long-distance pipelines (e.g., 10 Gbps core links with a 100 ms RTT), maintaining high throughput requires massive congestion windows ($W \approx 83,333$ segments). Using standard AIMD formulas, a 10 Gbps link can only tolerate an exceptionally low packet loss rate (L):

$$\text{Average Throughput} \approx \frac{1.22 \cdot \text{MSS}}{\text{RTT} \cdot \sqrt{L}}$$

To sustain 10 Gbps, the segment loss probability must be less than 2×10^{-10} (no more than 1 dropped segment out of every 5 billion packets). This limitation has prompted ongoing research into high-speed transport variants like BIC, CUBIC, and TCP Vegas.

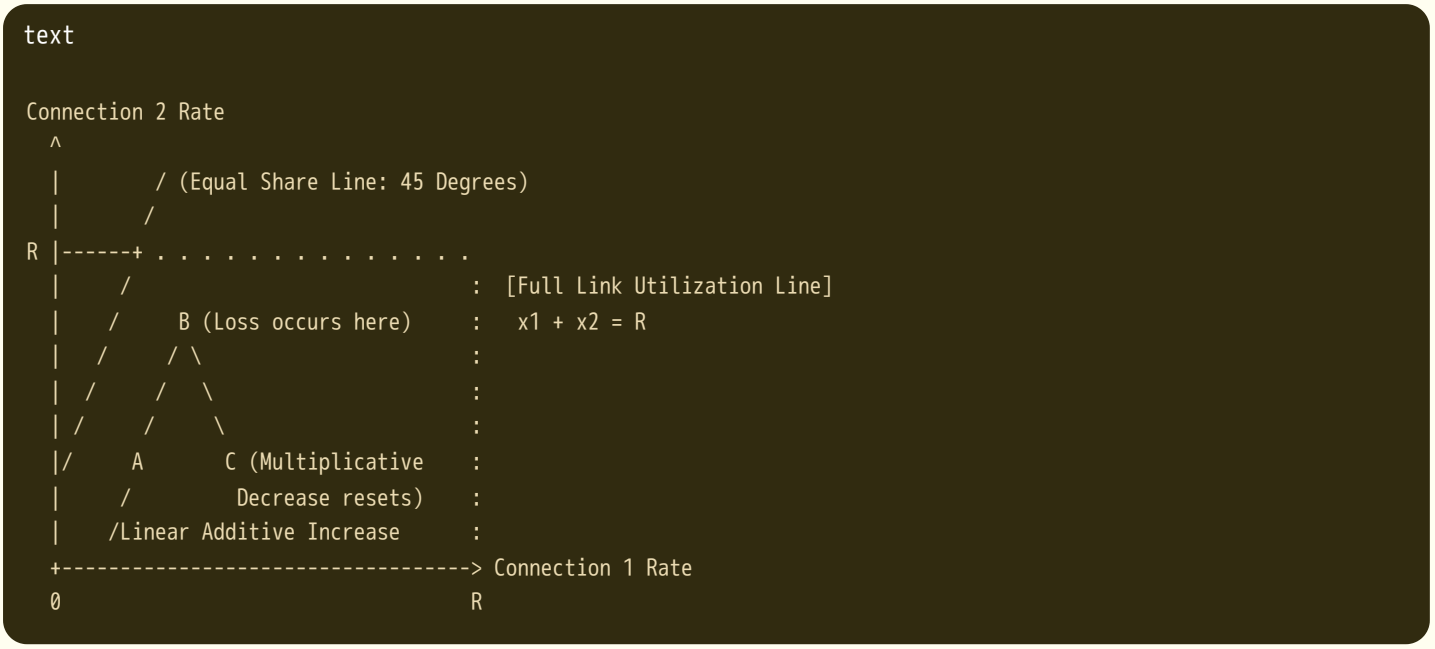
---

6. The Fairness Principle

A congestion control protocol is considered **fair** if K connections sharing a common bottleneck link of capacity R each achieve an equal throughput share of approximately R/K.

Why AIMD Converges to Fairness

Consider two connections competing for a shared link of capacity R:



1. **Additive Increase**: When the joint throughput lies below the full utilization line (Point A), no losses occur. Both sessions increase their windows by 1 MSS per RTT, moving their throughput along a 45-degree trajectory parallel to the fairness line.
2. **Multiplicative Decrease**: When the combined traffic exceeds link capacity (Point B), a loss event occurs. Both sessions halve their windows, shifting their throughput along a vector pointing directly toward the origin (Point C).
3. **Convergence Loop**: Because the session with the larger throughput drops by a larger absolute value during multiplicative decrease, this repeating cycle naturally drives both connections toward the equal share line.

Real-World Fairness Discrepancies

- **RTT Bias**: Connections with shorter physical paths complete their RTT loops faster, allowing them to open their congestion windows more aggressively than connections with longer paths.
- **Parallel TCP Connections**: Applications like Web browsers can open multiple parallel connections to a destination. If a link supports 9 applications using 1 connection each, a new application that opens 11 parallel sessions will claim over

half the total link bandwidth, creating an unfair allocation.

---

7. Explicit Congestion Notification (ECN)

To transition from pure end-to-end inference to a **network-assisted strategy**, modern deployments use **Explicit Congestion Notification (ECN)**.

- **Network Layer Routing**: Routers monitor internal buffer thresholds. If persistent congestion develops, the router marks a specialized 2-bit **Type of Service (TOS)** field in the IP header of packets passing through, rather than dropping them.
- **Receiver Reflection**: When the destination host receives this marked packet, its transport layer reads the indicator and sets the **ECE (ECN Echo)** flag bit in its next outgoing TCP ACK segment.
- **Sender Mitigation**: Upon receiving the ECE ACK, the sender halves its congestion window—reacting to the notification just as it would to a fast retransmit loss event. It then sets the **CWR (Congestion Window Reduced)** flag bit in the header of its next data segment to acknowledge the rate reduction.

Chapter Summary of Key Protocols and Variants

Protocol	Strategy	Key Characteristics
TCP Tahoe	GBN/SR Hybrid	Resets `cwnd = 1` on any loss event; slow starts to `ssthresh`.
TCP Reno	Modern Baseline	Implements Fast Recovery; halves window on triple duplicate ACKs.
TCP Vegas	Delay-Based	Tracks RTT growth to predict buffer bloat and gracefully backs off before drops occur.
QUIC	Application over UDP	Combines connection and security handshakes; bypasses kernel-level TCP bottlenecks.

This concludes the topic-by-topic notes for Chapter 3: Transport Layer. Please let me know if you would like to summarize specific review questions, explore programming assignments, or move on to another chapter!

Gemini 3.5 Flash Extended

Topic 3.8: Comprehensive Chapter Synthesis and Key Equations

---

1. Unified Protocol Header Synthesis

To review the transport layer, we compare the structural overhead of the two primary Internet transport protocols: **UDP** (Connectionless, 8 bytes) and **TCP** (Connection-Oriented, minimum 20 bytes).

```
text

UDP Header (8 Bytes Total)
+------------------------+------------------------+
|   Source Port (16b)    |  Destination Port (16b) |
+------------------------+------------------------+
|      Length (16b)      |    Checksum (16b)      |
```

```
+------------------------+------------------------+

TCP Header (Minimum 20 Bytes Total)
+------------------------+------------------------+
|   Source Port (16b)    |  Destination Port (16b) |
+------------------------+------------------------+
|                Sequence Number (32b)             |
+--------------------------------------------------+
|             Acknowledgment Number (32b)          |
+------------------------+------------------------+
| H.Len | Unused  | Flags |   Receive Window (16b)  |
+------------------------+------------------------+
|     Checksum (16b)     |  Urgent Pointer (16b)   |
+------------------------+------------------------+
|                   Options (Variable)             |
+--------------------------------------------------+
```

---

2. Comprehensive Mathematical Formula Sheet

Below is a compilation of the core equations utilized throughout Chapter 3 to compute network state, timeouts, utilization, and throughput.

Reliable Data Transfer & Pipelining

- **Stop-and-Wait Sender Utilization (U_{sender}):** $U_{\text{sender}} = \frac{d_{\text{trans}}}{\text{RTT} + d_{\text{trans}}} = \frac{L/R}{\text{RTT} + L/R}$
- **Pipelined Sender Utilization ($U_{\text{sender, pipelined}}$) for a window size of N:** $U_{\text{sender, pipelined}} = \frac{N \cdot d_{\text{trans}}}{\text{RTT} + d_{\text{trans}}} = \frac{N \cdot (L/R)}{\text{RTT} + L/R}$
- **Selective Repeat (SR) Window Size Constraint**: To prevent sequence number overlap and ambiguity between new packets and old retransmissions: $N_{\text{window}} \leq \frac{1}{2} \cdot \text{Sequence Number Space Size}$

TCP RTT Estimation & Timeout Calculation

- **Estimated RTT (Exponential Moving Average)**: $\text{EstimatedRTT}_n = (1 - \alpha) \cdot \text{EstimatedRTT}_{n-1} + \alpha \cdot \text{SampleRTT}_n \quad (\text{Recommended } \alpha = 0.125)$
- **Dev RTT (RTT Deviation)**: $\text{DevRTT}_n = (1 - \beta) \cdot \text{DevRTT}_{n-1} + \beta \cdot |\text{SampleRTT}_n - \text{EstimatedRTT}_n| \quad (\text{Recommended } \beta = 0.25)$
- **Timeout Interval**: $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$

TCP Flow Control

- **Receiver Available Window (rwnd):** $\text{rwnd} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$
- **Sender Transmission Constraint**: $\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$

TCP Congestion Control

- **Macroscopic Steady-State Throughput (AIMD Sawtooth)**: $\text{Average Throughput} = 0.75 \cdot \frac{W}{\text{RTT}} \text{ bytes/sec} \quad (\text{where } W \text{ is window size at loss event})$
- **High-Speed Link Loss-Throughput Relationship**: $\text{Average Throughput} \approx \frac{1.22 \cdot \text{MSS}}{\text{RTT} \cdot \sqrt{L}} \quad (\text{where } L \text{ is the packet loss probability})$

---

3. Pipelining Paradigms: GBN vs. Selective Repeat

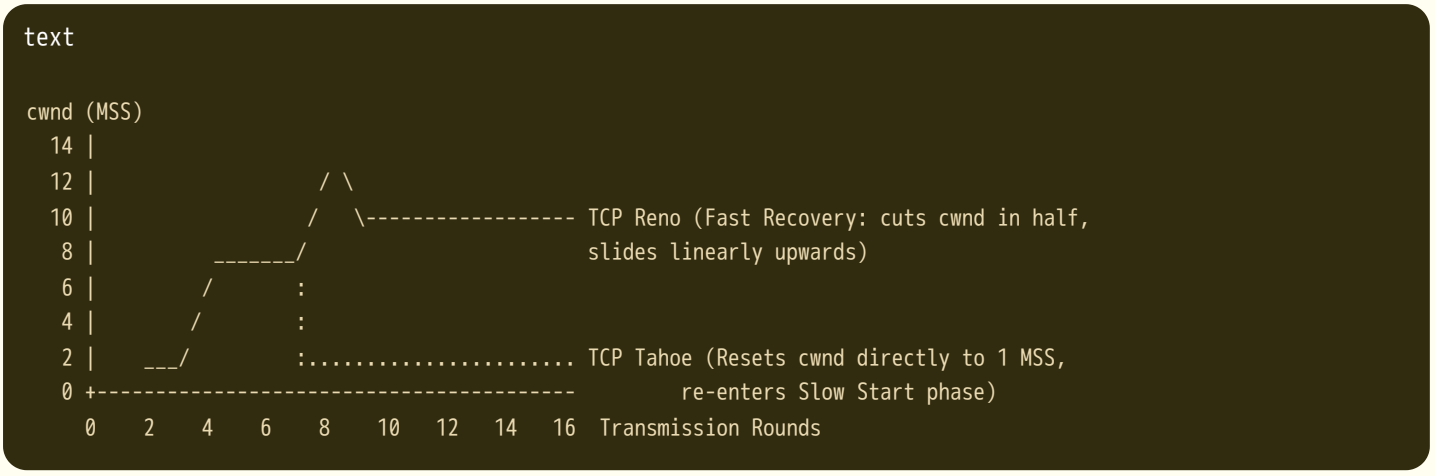
When recovery is required in a high-speed pipeline, **Go-Back-N** and **Selective Repeat** represent two distinct design philosophies.

Feature / Property	Go-Back-N (GBN)	Selective Repeat (SR)
Window Component	Single contiguous sender window.	Both sender and receiver maintain separate windows.
Acknowledgments	**Cumulative**: Validates all segments up to index n.	**Individual**: Validates the single specific packet received.
Timer Implementation	**Single Timer**: Anchored strictly to the oldest unacknowledged packet (`base`).	**Multiple Timers**: Every individual transmitted packet tracks its own logical timeout.
Out-of-Order Packets	**Discarded**: The receiver refuses to buffer fragments out of sequence and re-ACKs the last in-order packet.	**Buffered**: Saved in temporary storage until missing sequence gaps are successfully filled.
Retransmission Action	Retransmits the **entire active window** of unacknowledged packets upon timeout.	Retransmits **only** the single specific packet that experienced a drop or corruption.

---

4. Evolution of Congestion Control: Tahoe vs. Reno

The response curve of the congestion window (cwnd) illustrates the core optimization introduced by **TCP Reno** over the legacy **TCP Tahoe** protocol.



- **Shared Reaction**: For both Tahoe and Reno, a **Retransmission Timeout (RTO)** results in identical behavior: `ssthresh` is cut in half ($\text{cwnd}/2$), `cwnd` is reset to 1 MSS, and the connection drops back into the exponential **Slow Start** phase.
- **Divergent Reaction**: When **3 Duplicate ACKs** arrive:
  - **Tahoe** treats it like a timeout, dropping `cwnd` to 1 MSS.
  - **Reno** skips the full drop, executing a **Fast Retransmit**, cutting the window in half to the new `ssthresh`, inflating it by $+3$ MSS temporarily, and entering **Fast Recovery** before resuming linear **Congestion Avoidance**.

---

5. Architectural Map of the Transport Journey

This completes the structured overview of Chapter 3. We have traced how basic process-to-process multiplexing maps application streams onto an unreliable IP network, how sliding-window dynamics ensure reliable cross-planet delivery, and how self-clocking AIMD algorithms protect the global Internet infrastructure from systemic collapse.

All topics for Chapter 3: Transport Layer are now complete.